

Keerthana's Immediate List of Blockers:

Table of Contents

- (*SEV – 1* | *SEV – 2* | *SEV – 2.5* | *SEV – 3*)
- *Details and Explanation after Table-of-Contents.*
- *See the **Ultimate Production Checklist Doc** as my (KP's) support document to feed as pre-context to AI code-building agents.*

A. Sev–1 : Stop-Ship

a. consumer.go

1. Build blocker: unused `crypto/subtle import: preflight go build / go test ./...` failure
2. Plaintext password stored in projection: *credential exposure*
3. Raw password written to immutable event log: *permanent secret leakage*
4. Reset token path lacks secure token lifecycle: *timing, replay, expiry, single-use, scope, & attempt-limit gaps*
5. Ack after failed processing: *event loss*
6. No idempotency/dedupe: *duplicate delivery corrupts state*
7. No per-account serialization: *out-of-order projection updates*
8. No optimistic concurrency: *stale writes overwrite newer state*
9. Ignored JSON decode errors: *malformed events mutate state*
10. Unknown event types advance version: *projection drift*
11. Password reset event lacks version: *broken aggregate sequencing*
12. Timestamp-derived event ID: *unsafe retry/idempotency behavior*
13. Blind projection version assignment: *stale/gap/duplicate versions accepted*
14. Non-deterministic replay timestamp: *replay mismatch*

b. risk_scorer.py

1. Model evaluated on training data: *invalid production metric*
2. Accuracy used for rare compromise detection: *misleading risk score*
3. Full-history features likely leak future data: *invalid offline evaluation*
4. Scoring crashes for unseen accounts: *production scoring failure*
5. Raw account/IP logging: *PII leakage*

B. Sev-2 : Must Fix Before QA / Prod-Like Testing

a. consumer .go

1. One goroutine per event: *unbounded resource usage*
2. Missing cancellation behavior: *unsafe shutdown*
3. No timeout/retry/backoff/jitter: *unreliable failure handling*
4. Opaque ack behavior: *invisible ack failures*
5. Unbounded IP history: *memory/storage growth*
6. Invalid IP accepted: *bad state & bad features*
7. No password-reset rate limits: *abuse/brute-force risk*
8. No idempotency key for reset mutation: *duplicate event writes*
9. Missing event metadata: *weak event contract*
10. Missing request/header validation: *tenant/auth/schema risk*
11. Weak logs: *poor incident traceability*
12. Weak store interface: *cannot enforce event-sourcing semantics*
13. No schema versioning/migrations: *replay compatibility risk*
14. No DLQ/quarantine: *poison event risk*
15. No rate-limit header handling: *retry storms*
16. No severity telemetry: *weak incident response*
17. No supply-chain controls: *CVE/license/version risk*
18. No CI coverage gates: *weak release control*

b. risk_scorer.py

1. Full-history load into memory: *not scalable*
2. Oversimplified impossible travel: *weak fraud feature*
3. Implicit feature ordering: *schema fragility / training-serving skew risk*
4. Missing-label crash risk: *brittle training*
5. No random seed/config: *non-reproducible model*
6. No model governance: *unsafe ML release*
7. No scoring schema validation: *live crash/mis-score risk*
8. Empty-history default gap: *cold-start / feature-store / future scoring fragility*
9. Assumes `predict_proba` class order: *wrong risk probability*
10. No scoring request/header/rate-limit contract: *overload/traceability risk*
11. No Python/ML supply-chain controls: *CVE/license/reproducibility risk*
12. No ML observability/drift telemetry: *undetected model degradation*

C. Sev-2.5 : High Priority

a. consumer.go

1. Standard logger only: *weak structured logging*
2. Unsafe concurrency implementation despite correct intent: *scalability bug*
3. Ignored JSON marshal errors: *low-likelihood event-construction error handling gap*

b. risk_scorer.py

1. Dependency version not visible: *reproducibility risk*
2. `print("ship it")` : *unsafe release signal*
3. Timestamp sorting is good but incomplete: *point-in-time gap*

D. Sev-3 : Cleanup / Positive Patterns To Preserve

a. consumer.go

- 1. Event struct is readable but under-specified: *metadata gap*
- 2. Store abstraction is useful but incomplete: *contract gap*

b. risk_scorer.py

- 1. Feature extraction is compact but not production-safe: *refactor candidate*
- 2. Grouping by account is clear but not scalable: *baseline-only pattern*

E. Condensed Production Decision

- 1. **Do not merge :**
 - *production blockers remain*
- 2. **Do not promote to normal QA :**
 - *use QA only for failure reproduction unless blockers are remediated*

DETAILS & EXPLANATION

A. Sev-1 : Stop-Ship

a. consumer.go

Sl.#	Issue & Ln #	Blocker (TI;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
1.	<i>Ln:</i> 10 Build blocker: unused crypto/subtle import	go build / go test ./... fails before runtime review.	Remove the import or actually use subtle.ConstantTimeCompare .	This is more immediate than runtime findings because the submitted Go file should not pass preflight build. Refs : go.dev/unused-imports , pkg.go.dev/crypto/subtle

Sl.#	Issue & Ln #	Blocker (TI;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
2.	<i>Ln: 29-36, 89-92</i> Plaintext password stored in projection via PasswordPlain	Direct credential exposure in an identity system.	Remove PasswordPlain . Store only approved password-hash metadata.	Test plaintext never appears in state, logs, traces, or events. Refs : cheatsheetseries.owasp.org/password-storage , cwe.mitre.org/CWE-256
3.	<i>Ln: 101-109</i> Raw password written into immutable event payload as NewPassword	Permanently pollutes append-only audit log with credentials.	Never place passwords, tokens, or secrets in events. Append safe metadata only.	Add event-schema forbidden-field tests. Add secret scanning for generated event fixtures. Refs : cheatsheetseries.owasp.org/logging
4.	<i>Ln: 101-103</i> Reset token path lacks secure token lifecycle	Timing/replay risk; expiry, single-use, scope, account binding, & attempt-limit gaps remain.	Store hashed reset tokens. Use constant-time compare. Enforce expiry, single-use, scope, account binding, & attempt limits.	crypto/subtle is imported but unused. Tests : expired token, reused token, wrong account, attempt throttling. Refs : pkg.go.dev/crypto/subtle , cheatsheetseries.owasp.org/forgot-password
5.	<i>Ln: 62-65</i> Broker ack(e) runs even when apply fails	Failed events can be permanently lost.	Ack only after durable success. Retry, nack, or DLQ on failure.	Tests : LoadState failure, SaveState failure, validation failure must not ack success.
6.	<i>Ln: 56-66, 71-96</i> No idempotency/dedupe despite at-least-once delivery	Duplicate delivery corrupts counters, IP history, password state, & versions.	Track processed Event.ID atomically or enforce version-idempotency.	Tests : duplicate LoginFailed does not increment twice. Duplicate LoginSucceeded does not append twice.
7.	<i>Ln: 56-66, 71-96</i> No per-account serialization	Same-account events can apply out of order.	Partition/serialize workers by account/aggregate key.	Add race tests & same-account ordering tests. Refs : go.dev/race-detector
8.	<i>Ln: 39-43, 94-96</i> SaveState has no optimistic concurrency / expected-version check	Stale projection writes can overwrite newer account state.	Change API to SaveState(ctx, state, expectedVersion) with CAS/transaction semantics.	Tests : stale write, gap version, duplicate version, concurrent same-account update.
9.	<i>Ln: 82-91</i> json.Unmarshal errors ignored	Malformed payloads silently mutate state with zero values.	Check all decode errors. Validate schema, required fields, field types, & payload sizes.	Tests : malformed JSON, missing fields, wrong types, empty IP/password. Refs : pkg.go.dev/encoding/json

Sl.#	Issue & Ln #	Blocker (TI;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
10.	<i>Ln: 80-92, 94-96</i> Unknown event types silently advance projection version	Projection can skip required state changes while moving forward.	Add default case. Reject/quarantine unknown types. Do not save as success.	Tests : unknown type does not advance version. DLQ/quarantine path emits telemetry.
11.	<i>Ln: 104-109</i> Password reset appends PasswordChanged with no Version	Breaks aggregate sequencing & deterministic replay.	Append via event store that atomically assigns next aggregate version.	Tests : reset emits correct next version. Zero-version mutation rejected.
12.	<i>Ln: 105</i> Event ID generated from timestamp	Unsafe for retries/idempotency; collision & replay ambiguity.	Use UUID/ULID or idempotency-key-derived event ID.	Tests : retry same request uses same idempotency key/event result.
13.	<i>Ln: 94</i> Projection version assigned blindly from event	Allows stale, duplicate, zero, or gap versions.	Validate <code>event.Version == state.Version + 1</code> , except safe duplicate no-op.	Tests : stale, gap, duplicate, & zero-version handling.
14.	<i>Ln: 95</i> <code>time.Now()</code> mixed into replayed projection	Same event stream can rebuild to different state.	Separate deterministic domain state from operational rebuild metadata.	Replay same stream twice must produce identical domain projection. Refs : pkg.go.dev/time

b. risk_scorer.py

Sl.#	Issue & Ln #	Blocker (TI;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
1.	<i>Ln: 60-70</i> Model trains & evaluates on same data	Training accuracy is invalid for production readiness.	Use time-based train/validation/test split.	CI must reject training-set-only metrics. Refs : scikit-learn.org/model-evaluation , scikit-learn.org/train-test-split
2.	<i>Ln: 7-8, 67-69</i> Accuracy used for rare compromise detection	Always-negative classifier can look excellent when compromise rate is under 1%.	Use PR-AUC, recall at fixed FPR, precision@K, calibration, & threshold curves.	Add rare-class baseline & threshold tests. Refs : scikit-learn.org/precision-recall

Sl.#	Issue & Ln #	Blocker (TI;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
3.	<i>Ln: 45-57, 73-76</i> Full-history features likely leak future data	Offline training may use events unavailable at scoring time.	Build point-in-time features with event-time cutoff.	Tests : no post-score events in feature window. Label window starts after feature window.
4.	<i>Ln: 73-76</i> Scoring crashes for unseen accounts	New/sparse accounts are normal production cases.	Add cold-start defaults & safe fallback score.	Tests : missing account, missing fields, empty history, sparse history.
5.	<i>Ln: 49</i> Logs account ID & raw IP history	PII leakage into app logs.	Remove raw IP/account logging. Use aggregates, hashes, or redaction.	Add log-redaction tests. Refs : docs.python.org/logging , cheatsheetseries.owasp.org/logging

B. Sev-2 : Must Fix Before QA / Prod-Like Testing

a. consumer.go

Sl.#	Issue & Ln #	Blocker (TI;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
1.	<i>Ln: 54-66</i> One goroutine per event	Unbounded concurrency can exhaust memory, DB connections, FDs, & CPU.	Use bounded worker pool, semaphores, queues, & backpressure.	Burst-load test must assert max concurrency. Refs : pkg.go.dev/sync
2.	<i>Ln: 56-66</i> No explicit cancellation behavior inside processing loop	Shutdown may continue work or ack unexpectedly.	Check <code>ctx.Done()</code> before scheduling & inside workers.	Test graceful shutdown with in-flight events. Refs : pkg.go.dev/context
3.	<i>Ln: 71-96</i> No timeout/retry/backoff/jitter policy	Transient failures can cause loss, storms, or stuck work.	Add deadlines, retry budget, capped exponential backoff + jitter.	Fake-clock tests for timeout/backoff. Refs : rfc-editor.org/RFC-9110-Retry-After

Sl.#	Issue & Ln #	Blocker (TI;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
4.	<i>Ln: 62-65, 115</i> ack is opaque & not error-aware	Ack failures are invisible.	Make ack return error. Handle retry/metric/log policy.	Test ack failure path & broker redelivery behavior.
5.	<i>Ln: 85</i> LastSeenIPs grows unbounded	Memory/storage growth & duplicate amplification.	Bound, dedupe, TTL, or move to normalized/audit feature store.	Tests : IP cap, dedupe, history compaction.
6.	<i>Ln: 83-85</i> Empty/invalid IP accepted	Bad projection & bad model features.	Validate IP format & required payload fields.	Payload validation tests for empty/malformed IP.
7.	<i>Ln: 101-112</i> Password reset lacks rate limiting	Enables reset abuse & brute-force attempts.	Add per-account, per-IP, per-tenant, per-token attempt limits.	Tests for throttling/429 behavior. Refs : rfc-editor.org/RFC-6585-429
8.	<i>Ln: 101-112</i> No idempotency key for reset mutation	Retries can append duplicate password-change events.	Require idempotency key & persist replayed result.	Same key/same payload returns original result. Same key/different payload rejected.
9.	<i>Ln: 18-26</i> Event lacks explicit schema/content metadata	Weak long-term compatibility for append-only events.	Add schema version, content type, producer, trace/correlation fields.	Contract tests with golden events.
10.	<i>Ln: 18-26, 71-96</i> No request/header metadata validation implied	Tenant, partition, auth, content-type, idempotency, trace context cannot be verified.	Validate required API/broker envelope fields before append/apply.	Tests : missing tenant, mismatched account, unsupported content type, missing schema version.
11.	<i>Ln: 62-65</i> Logs only event ID & error	Insufficient incident traceability.	Use structured logging with trace ID, event ID, account hash, tenant, partition, severity.	Confirm no secrets/PII in logs. Refs : cheatsheetseries.owasp.org/logging
12.	<i>Ln: 39-43</i> Store interface too weak for event sourcing	Cannot express CAS, dedupe, transactions, DLQ, or idempotency.	Extend store API for atomic append, save-with-version, processed-event markers.	Integration tests with conflict simulation.

Sl.#	Issue & Ln #	Blocker (TI;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
13.	<i>Ln: 80-92</i> No schema versioning/migrations for event payloads	Old/new permanent events may not replay safely.	Versioned event schemas & compatibility policy.	Golden replay fixtures for old versions.
14.	<i>Ln: 71-96</i> No DLQ/quarantine path	Poison events can loop or be dropped.	Route invalid/non-retryable events to DLQ with metadata.	Tests for malformed/unknown events to DLQ.
15.	<i>Ln: All</i> No downstream rate-limit handling	429/503 can trigger retry storms or event loss.	Honor <code>Retry-After</code> , <code>X-RateLimit-*</code> ; add adaptive throttling & circuit breakers.	Fake server tests for 429/503 & headers. Refs : rfc-editor.org/RFC-9110-Retry-After , rfc-editor.org/RFC-9333-RateLimit
16.	<i>Ln: All</i> No Sev-1/2/2.5/3 telemetry model	Incidents cannot be routed or triaged reliably.	Metrics & alerts for event loss, DLQ, lag, rate limits, validation failures, projection conflicts.	Alert tests for each severity.
17.	<i>Ln: All</i> No supply-chain controls shown	Enterprise prod needs CVE, version, package, & license governance.	Pin deps/images; generate SBOM; run CVE/license scans; track EOL.	Block critical CVEs, unknown licenses, & prohibited licenses. Refs : cyclonedx.org/SBOM , spdx.org/licenses , nvd.nist.gov
18.	<i>Ln: All</i> No CI coverage gates shown	Critical AI-generated code lacks release protection.	Require unit/validation/regression/integration tests & 80-85% coverage.	Add changed-line coverage, race detector, lint, secret scan.

b. risk_scorer.py

Sl.#	Issue & Ln #	Blocker (TI;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
1.	<i>Ln: 19-24, 79-82</i> Loads full event history into memory	Not scalable for long histories & hundreds of thousands of accounts.	Use streaming/partitioned reads, incremental features, feature store.	Large-history memory/perf tests.
2.	<i>Ln: 39-42</i> <code>detect_impossible_travel</code> is oversimplified	Country-count heuristic is not actual impossible travel.	Use geo distance/time deltas & validated feature definition.	Feature-review test & doc.

Sl.#	Issue & Ln #	Blocker (Tl;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
3.	<i>Ln: 61, 76</i> Implicit feature ordering via <code>dict.values()</code>	Schema fragility / training-serving skew risk if feature construction changes.	Define <code>FEATURE_COLUMNS</code> ; vectorize by explicit schema.	Schema compatibility test. Refs : docs.python.org/dict
4.	<i>Ln: 62</i> Label lookup can crash on missing label	Training can fail or create biased workaround risk.	Validate label coverage; define unlabeled-account policy.	Tests for missing/extra labels.
5.	<i>Ln: 64</i> <code>RandomForestClassifier()</code> has no seed/config	Non-reproducible model artifacts.	Set <code>random_state</code> ; store config/code/data versions.	Deterministic training test. Refs : scikit-learn.org/RandomForestClassifier
6.	<i>Ln: 64-70</i> No model artifact governance	No model version, registry, approval, rollback, threshold record.	Store model version, feature schema, metrics, threshold, data snapshot.	Release gate requires model card/approval.
7.	<i>Ln: 73-76</i> No event schema validation at scoring	Bad live events can crash or mis-score.	Validate required fields & types before scoring.	Tests : missing <code>account_id</code> , malformed event.
8.	<i>Ln: 45-57</i> Missing empty-history defaults	Not reachable from current grouped training path, but will break cold-start accounts, feature-store rows, or future scoring paths.	Define explicit empty-history defaults.	Unit test empty event list/account. Keep this lower priority than the existing unseen-account scoring crash.
9.	<i>Ln: 76</i> Assumes <code>predict_proba(...)</code> <code>[0][1]</code> maps to positive class	Wrong class order can return wrong risk probability.	Use <code>clf.classes_</code> to locate positive class index.	Test class-order handling. Refs : scikit-learn.org/predict_proba
10.	<i>Ln: All</i> No request/header/rate-limit contract for live scoring path	Scoring can be overloaded or lose traceability.	Require request ID, trace ID, tenant/account metadata, schema version, & idempotency where applicable.	Metrics for scoring latency, throttling, missing headers.

Sl.#	Issue & Ln #	Blocker (TI;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
11.	<i>Ln:</i> All No supply-chain controls for Python/ML deps	Unpinned ML deps can create CVE, license, reproducibility, & migration risk.	Pin Python/sklearn versions; lock deps; SBOM; CVE/license scan.	Block unknown licenses & critical CVEs. Refs : pip.pypa.io/requirements , packaging.python.org/dependency-specifiers
12.	<i>Ln:</i> All No ML observability/drift telemetry	Live model degradation cannot be detected.	Emit score distribution, feature drift, missing feature rate, threshold crossings, latency.	Drift alerts & shadow-mode metrics.

C. Sev-2.5 : High Priority

a. consumer.go

Sl.#	Issue & Ln #	Blocker (TI;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
1.	<i>Ln:</i> 13, 63 Standard logger only, no structured logging	Poor prod diagnostics & weak log governance.	Use structured logger with redaction & standard fields.	Include trace/event/tenant IDs. Verify no secrets.
2.	<i>Ln:</i> 54-55 Comment identifies throughput issue but implementation is unsafe	Good intent, poor concurrency model.	Replace goroutine-per-event with partition-aware bounded worker pool.	Keep goal: slow account should not block unrelated accounts.
3.	<i>Ln:</i> 103 json.Marshal errors are ignored	In this exact payload shape, failure is unlikely, but event construction should still be checked before append.	Check marshal errors before append. Prefer typed event encoders.	Add error-handling coverage for event construction paths. Refs : go.dev/error-handling

b. risk_scorer.py

Sl.#	Issue & Ln #	Blocker (TI;Dr)	Mitigate/Resolve	Notes/tests/long-term fixes
1.	<i>Ln:</i> 14 Dependency version not visible	Reproducibility, CVE, & license posture unclear.	Pin scikit-learn & transitive deps in lockfile.	Add dependency/CVE/license scan.

Sl.#	Issue & Ln #	Blocker (<i>TI;Dr</i>)	Mitigate/Resolve	Notes/tests/long-term fixes
2.	<i>Ln:</i> 67-69 print("ship it") release signal	Unsafe promotion signal for security ML.	Remove print-based approval. Use formal release gates.	CI should fail on training-only approval language.
3.	<i>Ln:</i> 47-48 Events sorted by timestamp before feature extraction	Good local ordering habit but insufficient point-in-time safety.	Preserve sorting but add cutoff-aware feature generation.	Positive pattern to keep, with leakage tests.

D. Sev-3 : Cleanup / Positive Patterns To Preserve

a. consumer.go

Sl.#	Issue & Ln #	Blocker (<i>TI;Dr</i>)	Mitigate/Resolve	Notes/tests/long-term fixes
1.	<i>Ln:</i> 18-26 Event struct is readable but under-specified	Good start, missing enterprise metadata.	Keep typed event struct. Add schema version, content type, trace ID, tenant/partition metadata.	Positive pattern: explicit event object.
2.	<i>Ln:</i> 39-43 Store abstraction improves testability but lacks production semantics	Useful interface boundary, incomplete contract.	Preserve abstraction. Extend for CAS, idempotency, DLQ, transactions.	Positive pattern: side effects behind interface.

b. risk_scorer.py

Sl.#	Issue & Ln #	Blocker (<i>TI;Dr</i>)	Mitigate/Resolve	Notes/tests/long-term fixes
1.	<i>Ln:</i> 45-57 Feature extraction is compact & reviewable	Good structure, but not production-safe yet.	Keep small feature-builder shape. Make it schema-bound, cutoff-aware, & scalable.	Positive pattern for assistant skill memory.
2.	<i>Ln:</i> 32-36 Grouping by account is simple & clear	Useful baseline organization, not scalable alone.	Replace with streaming/partitioned grouping for prod scale.	Positive pattern for small tests/fixtures.

E. Condensed Production Decision

Do not merge or promote this code to production.

Do not push to normal QA as-is unless QA is explicitly being used for failure reproduction.

Primary code blockers

- 1. Credential handling is unsafe.
- 2. Immutable event log can be polluted with secrets.
- 3. Failed events can be acknowledged & lost.
- 4. At-least-once delivery is not handled idempotently.
- 5. Same-account ordering is not preserved.
- 6. Projection writes are not concurrency-safe.
- 7. Event schemas are not validated/versioned.
- 8. Risk model evaluation is invalid.
- 9. Accuracy is unsuitable for rare compromise detection.
- 10. Feature generation likely leaks future data.
- 11. Scoring path is brittle for unseen accounts.
- 12. Logs can leak PII.

Production hardening before release

- 1. Supply-chain, SBOM, package-version, CVE, & license controls.
- 2. CI coverage gates across unit, validation, regression, & integration tests.
- 3. Severity telemetry model.
- 4. Structured observability.
- 5. Request, tenant, & header contracts.
- 6. Rate limiting, throttling, backoff, & Retry-After / X-RateLimit handling.

END
